

清华大学本科生考试试题专页纸	
考试课程：操作系统（A）	时间：2024年06月16日上午 09:00~11:30

系别：	班级：	学号：	姓名：
-----	-----	-----	-----

答卷注意 事项：	1. 答题前，请先在试题纸和答卷本上写明A卷或B卷、系别、班级、学号和姓名。
	2. 在答卷本上答题时，要写明题号，不必抄题。
	3. 试卷有15道判断题、10道选择题、6道问答题。
	4. 请回答所有题目。本试卷共8页。最后空白页可做草稿。
	5. 考试完毕，必须将试题纸和答卷本一起交回。

一、(15分)判断题

- 1. ☐ 关于安全状态与死锁的关系：出现死锁的系统一定是不安全的。
- 2. ☐ 在采用索引结构文件系统的类UNIX操作系统（如Linux）中，通过软链接方式在链接目录时可能形成环形目录结构。
- 3. ☐ 多个进程同时读写一个文件可能导致并发冲突，因此编写并行程序时不应该出现多个进程同时打开一个文件的情况。
- 4. ☐ 在多核进程调度中的工作窃取机制有助于提高进程的缓存命中率。
- 5. ☐ 操作系统中无父子关系的两个进程可以通过命名管道机制进行通信。
- 6. ☐ 文件描述符用于标识由进程打开的文件，当用户进程访问open系统调用并成功返回时，它会得到一个大于等于0的文件描述符。
- 7. ☐ 父进程先于子进程退出会导致子进程变为僵尸进程。
- 8. ☐ 在实时操作系统中，面向周期性任务的RM调度算法（Rate Monotonic Scheduling）是一种静态优先级调度算法，周期越长的任务，其优先级越高。
- 9. ☐ 在类UNIX（如Linux）系统中，“一切皆文件”的设计哲学意味着打印机设备、系统内存使用情况和进程调度情况都可表示为文件。
- 10. ☐ 进程间间接通信的方式包括：信号、管道、使用共享内存和文件。
- 11. ☐ 在多线程编程中，原子操作能保证在同一时刻只有一个线程可以执行该操作。
- 12. ☐ 死锁是由于多个进程或线程各自持有一部分资源并请求其它资源，但其它资源已被对方持有，从而导致循环等待现象。
- 13. ☐ 管程（Monitor）是一种高级同步机制，它不允许进程在等待某个条件变量时释放管程的锁。
- 14. ☐ 信号SIGINT是由用户通过按下键盘上的“如Ctrl+C”组合键生成。进程可以设置信号处理函数来改变信号SIGINT的默认行为。
- 15. ☐ 使用信号量（Semaphore）机制可以实现进程间的同步，但信号（Signal）不能用于进程间同步。

二、(20分)单项选择题

- 1. mmap系统调用可以将文件内容映射至内存地址空间中，这样可以通过访问内存的方式随机访问文件内容。假设某个程序通过mmap成功地将部分文件内容映射到一段先前未使用的虚拟地址空间中，以下哪些行

为/事件一定会发生? ()

- A. 进程的文件描述符表中出现了新的一项
 - B. 发起磁盘I/O读取文件内容
 - C. 文件内容被复制到新的物理页
 - D. 修改该进程的页表
2. 某计算机系统中有 9 台图形扫描仪, 由 K 个进程竞争使用, 每个进程最多需要 4 台图形扫描仪。该系统可能会发生死锁的 K 的最小值是 ()
- A. 2
 - B. 3
 - C. 4
 - D. 5
3. 在基于索引方式的文件系统中, 设文件F1的当前引用计数值为1, 用户通过命令行操作, 先建立F1的符号链接(软链接)文件F2, 再建立F1的硬链接文件F3, 再建立F2的硬链接文件F4, 然后删除F1和F2。此时, F3和F4的引用计数值分别是 ()
- A. 1、1
 - B. 1、2
 - C. 2、1
 - D. 2、2
4. 下列命中组合情况中, 一次访存过程中不可能发生的是 ()
- A. TLB未命中, Cache未命中, Page Table未命中
 - B. TLB未命中, Cache命中, Page Table命中
 - C. TLB未命中, Cache未命中, Page Table命中
 - D. TLB命中, Cache命中, Page Table未命中
5. 下列寄存器中, 对用汇编语言和高级语言编写操作系统的程序员可见的是 ()
- A. 存储器地址寄存器 (MAR)
 - B. 存储器数据寄存器 (MDR)
 - C. 程序计数器 (PC)
 - D. 指令寄存器 (IR)
6. 设与某资源关联的信号量初值为3, 当前值为1。若M表示该资源的可用个数, N表示等待该资源的进程数, 则M、N分别是 ()
- A. 0、1
 - B. 1、0
 - C. 1、2
 - D. 2、0

7. 某计算机采用二级页表的分页存储管理方式, 按字节编址, 页大小为 2^{10} 字节, 页表项大小为2字节, 虚拟地址结构为:

页目录号	页号	页内偏移
------	----	------

虚拟地址空间大小为 2^{16} 页, 则表示整个虚拟地址空间的页目录表中包含表项的个数至少是 ()

- A. 64
 - B. 128
 - C. 256
 - D. 512
8. 设文件索引节点中有7个地址项, 其中4个地址项是直接地址索引, 2个

地址项是一级间接地址索引，1个地址项是二级间接地址索引，每个地址项大小为4字节。若磁盘索引块和磁盘数据块大小均为256字节，则可表示的单个文件最大长度是（ ）

- A. 33 KB
- B. 519 KB
- C. 1057 KB
- D. 16513 KB

9. 在缺页处理过程中，操作系统可能包含的执行操作是（ ）
I 修改页表 II 磁盘I/O III 分配页框 IV 异常处理 V 进程调度

- A. 仅I、II
- B. 仅I、II、III
- C. 仅I、II、III、IV
- D. I、II、III、IV、V

10. 在采用基于索引结构的文件系统的UNIX和类UNIX(如Linux)中，关于read系统调用的叙述，正确的是（ ）

- I read系统调用会产生磁盘写操作
- II read系统调用会导致CPU从用户态切换到核心态
- III read系统调用会检查提供的文件描述符是否对应一个已打开且有效的文件
- IV read系统调用可使用Copy on Write机制来优化读取性能
- V read系统调用在读取数据后需释放内存中的文件索引节点（inode）

- A. 仅I、II、III
- B. 仅II、III、IV
- C. 仅III、IV、V
- D. I、II、III、IV、V

三、(65分)共6道问答题

1. (10分)

阅读下面多线程求最大值的代码，回答问题：

```
1. #define thread_count (8)
2. int data[1000000], n;
3. int ans = INT_MIN; // INT_MIN表示int型变量能够存储的最小可能值
4. void *func(void *p) {
5.     int id = (long)p, local_result = INT_MIN;
6.     int slice = _____; // 请填写代码
7.     int start = id * slice;
8.     int stop = min(n, start + slice);
9.     for (int i = start; i < stop; i++)
10.         local_result = max(local_result, data[i]);
11.     ans = max(ans, local_result);
12.     return NULL;
13. }
14.
15. int main() {
16.     scanf("%d", &n);
17.     for (int i = 0; i < n; i++)
```

```

18. scanf("%d", data + i);
19. pthread_t threads[thread_count];
20. for (int i = 0; i < thread_count; i++)
21.     pthread_create(threads + i, NULL, func, (void *)(&i));
22. for (int i = 0; i < thread_count; i++)
23.     pthread_join(threads[i], NULL);
24. printf("ans: %d\n", ans);
25. return 0;
26. }

```

运行代码时，用户会先输入一个整数 n ，然后再输入 n 个整数，用户会保证这些整数均在 `int` 的表示范围之内。

(1) 为保证程序会不重不漏地遍历 `data` 数组的 n 个元素，请补全 Line 6 的横线上空缺的代码。

(2) 运行上述程序时至多会同时有多少线程？当它们同时存在时分别在完成什么工作？

(3) Line 21 调用 `pthread_create` 参数中的 `(void *)(&i)` 的目的是什么？

(4) 小明发现运行上述代码时会随机出现答案不正确的情况，即出现输出的 `ans` 小于实际的最大值的情况，请详细解释为何会出现这一错误，并且提出解决方案。

2. (10分)

小明在学习了CAS (Compare And Swap) 原子操作指令后，准备用它进行实战。现在有一个生产者-消费者模型的问题，若干生产者和若干消费者需要一个线程安全的结构作为任务队列，小明用C语言编写了如下的基于单链表的非阻塞线程安全栈的结构和操作（`compare_and_swap` 函数是 CAS 原子操作指令的封装），请阅读代码回答问题：

```

1. struct Node {
2.     Node *next;
3.     int value;
4. };
5.
6. struct Stack {
7.     Node *top;
8. };
9.
10. void init(Stack *s) {
11.     s->top = NULL;
12. }
13.
14. void push(Stack *s, Node *n) {
15.     while (1) {
16.         Node *old_top = s->top;
17.         _____; // 请填写代码
18.         if (compare_and_swap(&s->top, old_top, n))
19.             return;
20.     }

```

```

21. }
22.
23. Node *pop(Stack *s) {
24.     while (1) {
25.         Node *old_top = s->top;
26.         if (old_top == NULL)
27.             return NULL;
28.         _____; // 请填写代码
29.         if (compare_and_swap(&s->top, old_top, new_top))
30.             return old_top;
31.     }
32. }

```

(1) 请给出CAS (Compare And Swap) 原子操作指令的语义的伪代码实现和必要的注释。

(2) 请补全 Line 17 与 Line 28 的C代码以使得上述代码符合小明的需求。

(3) 小明编写的栈在上述生产者-消费者问题中符合期望地工作了，但是当小明把它交给助教做并行测试时，却被助教测出了BUG，出现了数据丢失的情况。**已知助教的测试方式并不符合生产者-消费者模型**，请举出一种可以导致BUG的具体情形并详细分析。

(4) 请简要说明一种能解决上述问题的办法。

3. (10分)

以下是课程实验中的相关代码片段，请在 rCore/uCore Tutorial 实验中选择其一回答下列问题。

// rCore Tutorial 实验选择

#[repr(C)]

```

pub struct DiskInode {
    pub size: u32,
    pub direct: [u32; INODE_DIRECT_COUNT],
    pub indirect1: u32,
    pub indirect2: u32,
    type_: DiskInodeType,
}

```

// uCore Tutorial 实验选择

```

struct dinode {
    short type;
    short pad[3];
    uint size;
    uint addrs[NDIRECT + 1];
};

```

(1) 小明想在 DiskInode / dinode 中添加一个 64bit 的成员变量用于记录文件创建时间。除了将其插入结构体之外，上述结构体还需要做什么修改？为什么？

(2) 如果在 struct inode 中插入同样的成员变量，是否也需要做一样的修改？为什么？

(3) 小明考虑到在 DiskInode / dinode 中插入的信息太多了（链接数、创建/修改/访问时间等），使得该结构体无法存下所有内容。故决定将该结构体的大小翻倍。假设硬盘上 DiskInode / dinode 所占的总空间不变，那么还需要在超级块 (SuperBlock) 和位图块 (BitMap) 做什么修改？

4. (8分)

- (1) 死锁发生的必要条件是什么？请列出这些条件并简要说明。
- (2) 考虑系统的以下快照（A--E是进程编号， U—X是资源类型）：

	Allocation	Max	Need	Available
	U V W X	U V W X	U V W X	U V W X
A	1 1 0 2	1 1 0 3	0 0 0 1	3 4 0 0
B	0 1 2 3	1 2 4 5	1 1 2 2	
C	5 1 4 4	5 5 9 5	0 4 5 1	
D	0 0 1 1	5 0 1 1	5 0 0 0	
E	1 0 2 1	4 3 2 1	3 3 0 0	

该系统处于安全状态吗？并请给出解释。

- (3) 考虑一个系统的以下快照（A--D是进程编号， U—X是资源类型）：

	Allocation	Need	Available
	U V W X	U V W X	U V W X
A	1 0 1 1	0 1 0 0	0 0 0 0
B	1 1 0 0	0 1 1 0	
C	1 0 1 1	0 0 0 0	
D	0 0 0 0	1 0 1 0	

系统处于死锁状态吗？如果系统不处于死锁状态，请给出解释。如果系统处于死锁状态，请列出所有涉及死锁的进程。

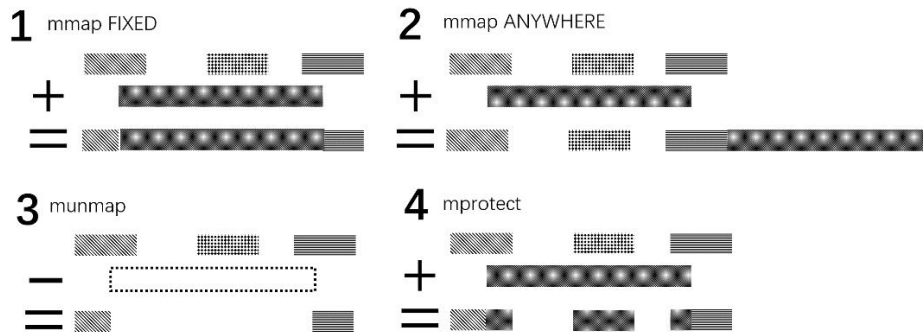
5. (15分)

三个线程P1、P2、P3互斥使用一个包含N(N>10)个单元的缓冲区。P1每次用produce()生成一个随机整数并用put()送入缓冲区某一空单元中；P2每次用getPos()从该缓冲区中取出一个大于等于零的整数并用countPos()统计这类整数的个数；P3每次用getNeg()从该缓冲区中取出一个小于零的整数并用countNeg()统计这类整数的个数。

- (1) 请用信号量机制实现这三个线程的同步与互斥活动。要求用伪代码描述，并给出清晰明了的注释（包含所定义信号量的含义和初值，以及必要的代码解释）。
- (2) 请用管程机制实现这三个线程的同步与互斥活动。要求用伪代码描述，并给出清晰明了的注释（包含所定义互斥锁和条件变量的含义，以及必要的代码解释）。

6. (12分)

在第四章实验中，提到了分配内存的 mmap 和取消内存分配的 munmap。而实际的内存分配操作可分为以下四类，如下图所示。



- mmap FIXED 表示在固定地址映射一段内存。如果它覆盖了一些原有的区间，那么原本的映射将被取代。
- mmap ANYWHERE 表示在任意位置映射一段固定长度的内存。
- munmap 表示取消一段内存区间中的映射。munmap 指定的区间不必原本都有映射。如果指定了区间的一部分，则需要切断、缩短原本的区间。
- mprotect 表示改变一段内存区间的权限(只读/只写/可读写等)。mprotect 指定的区间不必原本都有映射。如果指定了区间的一部分，则需要切断原本的区间，重新划分为两个区间。

在本题中，我们认为所有的区间操作都会成功，不需要考虑区间未对齐、区间长度为0或负数等非法情况。内核中已保存的所有区间互不相交，在下面代码操作中插入的区间也都应该互不相交。假设内核有以下函数可供你调用：

1. // 当 [start, end) 全部未被映射时，映射这段地址，令权限为 prot
2. void map(int start, int end, int prot);
3. // 当 [start, end) 全部被映射时，取消映射这段地址
4. void unmap(int start, int end);
5. // 获取地址 addr 所在页面的权限(prot)
6. int get_prot(int addr);
7. // 加入一个新区间 [start, end)，区间内页面权限为 prot
8. void add_area(int start, int end, int prot);
9. // 删除一个已有区间 [start, end)
10. void remove_area(int start, int end);

你需要从下列 A--H 选项中选择 5 项，分别填入以下代码的 (1)--(5) 处，以实现 mmap FIXED 功能。

- A. area_start < start && start < area_end;
- B. area_start < end && end < area_end;
- C. area_start <= end || start <= area_end;
- D. map(start, end, prot); add_area(start, end, prot);
- E. unmap(start, end); remove_area(start, end);
- F. map(area_start, area_end, prot); add_area(area_start, area_end, prot);
- G. unmap(area_start, area_end); remove_area(area_start, area_end);
- H. unmap(start, end); remove_area(area_start, area_end); add_area(area_start, start, get_prot(area_start)); add_area(end, area_end, get_prot(area_start));

需要补充完整的代码：

```

1. void mmap_fixed(int start, int end, int prot) {
2.     for (int i = 0; i < map_area_count; i++)
3.     { // 枚举每段当前已被映射的区间
4.         // 本次迭代枚举的区间
5.         int area_start = map_area[i].start;
6.         int area_end = map_area[i].end;
7.         if (start <= area_start && area_end <= end) {
8.             ____ (1) ____
9.         } else if (area_start <= start && end <= area_end) {
10.            ____ (2) ____
11.        } else if (____ (3) ____) {
12.            unmap(start, area_end);
13.            remove_area(area_start, area_end);
14.            add_area(area_start, start, get_prot(area_start));
15.            // add_area/remove_area 会被推迟到循环结束后再更新,
16.            // 因此不必担心它们会影响循环遍历的 map_area[i],
17.            // 且你也不需要手动修改 map_area_count
18.        } else if (____ (4) ____) {
19.            .....//这里有一些代码, 但不需要填写
20.        }
21.    }
22.    ____ (5) ____
23.    // 更新 add_area/remove_area 对区间所做的修改
24.    map_area_count = update();
25. }

```

(1) 请给出上述代码中 (1) -- (5) 要填写空所对应的 A-- H 中的选项。

(2) 若有下面的 **protect** 函数:

```

// 当 [start, end) 全部被映射时, 将区间内的页的权限修改为 prot
void protect(int start, int end, int prot);

```

(2.1) 它可以替换掉上一小问 (1) -- (4) 中哪一个空的所在分支中的实现?

(2.2) 用它替换掉原有实现的优点是什么? 给出解释。

(2.3) 如果 map_area 里保存的区间是无序的, 那么在上一小问 (1) -- (4) 中的所在分支, 哪些可以在结尾插入 “break;” 语句?

(2.4) 如果 map_area 里保存的区间是有序的, 按区间左端点地址从大到小排列, 那么在上一小问 (1) -- (4) 中的所在分支, 哪些可以在结尾插入 “break;” 语句?

